

Task 4:

Kamasutra.py:

```
1  #!/usr/bin/env python3
2  import sys
3  import random
4  import os
5
6  def generate_keypair():
7      """
8      Generates a random Kama Sutra key pairing (26 letters, each paired with another).
9      'f' is paired but will be skipped during encryption/decryption.
10     Returns a dictionary mapping each letter to its partner.
11     """
12     alphabet = list('abcdefghijklmnopqrstuvwxyz')
13     random.shuffle(alphabet)
14     pairs = {}
15
16     # Pair first half with second half
17     half = len(alphabet) // 2
18     first_half = alphabet[:half]
19     second_half = alphabet[half:]
20
21     for i in range(half):
22         a = first_half[i]
23         b = second_half[i]
24         pairs[a] = b
25         pairs[b] = a
26
27     # Ensure all letters are in the dictionary (should be 26 entries)
28     return pairs
```

```
30 def save_key(key, filename):
31     """Save key to a file in alphabet order for readability."""
32     with open(filename, 'w') as f:
33         for letter in 'abcdefghijklmnopqrstuvwxyz':
34             f.write(f"{letter}{key[letter]}\n")
35
36 def load_key(filename):
37     """Load key from file."""
38     key = {}
39     with open(filename, 'r') as f:
40         for line in f:
41             line = line.strip()
42             if len(line) == 2:
43                 a, b = line[0], line[1]
44                 key[a] = b
45                 key[b] = a
46     return key
47
48 def encrypt(plaintext, key):
49     """Encrypt plaintext using Kama Sutra cipher, skipping 'f'."""
50     ciphertext = []
51     for ch in plaintext:
52         if ch == 'f' or ch not in key:
53             ciphertext.append(ch) # 'f' is not replaced
54         else:
55             ciphertext.append(key[ch])
56     return ''.join(ciphertext)
```

```

58 def decrypt(ciphertext, key):
59     """Decrypt ciphertext (same as encrypt in this cipher)."""
60     return encrypt(ciphertext, key) # Symmetric cipher
61
62 def process_file(input_file, output_file, key, mode='encrypt'):
63     """Read input file, encrypt/decrypt, write to output file."""
64     try:
65         with open(input_file, 'r') as f:
66             text = f.read().strip().lower()
67     except FileNotFoundError:
68         print(f"Error: File '{input_file}' not found.")
69         sys.exit(1)
70
71     if mode == 'encrypt':
72         result = encrypt(text, key)
73     else: # decrypt
74         result = decrypt(text, key)
75
76     with open(output_file, 'w') as f:
77         f.write(result)
78
79 def main():
80     if len(sys.argv) < 2:
81         print("Usage:")
82         print("  kamasutra -k <keyfile.txt>")
83         print("  kamasutra -e <keyfile.txt> <plaintext.txt> <ciphertext.txt>")
84         print("  kamasutra -d <keyfile.txt> <ciphertext.txt> <plaintext.txt>")
85         sys.exit(1)
86

```

```

87     option = sys.argv[1]
88
89     if option == '-k' and len(sys.argv) == 3:
90         keyfile = sys.argv[2]
91         key = generate_keypair()
92         save_key(key, keyfile)
93         print(f"Key generated and saved to {keyfile}")
94
95     elif option == '-e' and len(sys.argv) == 5:
96         _, _, keyfile, plaintext, ciphertext = sys.argv
97         key = load_key(keyfile)
98         process_file(plaintext, ciphertext, key, 'encrypt')
99         print(f"Encryption complete. Ciphertext saved to {ciphertext}")
100
101     elif option == '-d' and len(sys.argv) == 5:
102         _, _, keyfile, ciphertext, plaintext = sys.argv
103         key = load_key(keyfile)
104         process_file(ciphertext, plaintext, key, 'decrypt')
105         print(f"Decryption complete. Plaintext saved to {plaintext}")
106
107     else:
108         print("Invalid arguments. Check usage.")
109         sys.exit(1)
110
111 if __name__ == '__main__':
112     main()

```

Makefile.txt:

```
# Makefile for Kamasutra cipher program

PYTHON = python3
SCRIPT = Kamasutra.py
TARGET = kamasutra

all: $(TARGET)

$(TARGET): $(SCRIPT)
    cp $(SCRIPT) $(TARGET)
    chmod +x $(TARGET)

clean:
    rm -f $(TARGET) *.txt Output.txt

test:
    ./$(TARGET) -k keyfile.txt
    ./$(TARGET) -e keyfile.txt Ptext-1.txt Ctext-3.txt
    ./$(TARGET) -d keyfile.txt Ctext-3.txt Output.txt
    diff Output.txt Ptext-1.txt && echo "Test passed!"

.PHONY: all clean test
```

PText-1.txt

```
Unfortunately critics say the components are often clued
into the process and have become integral factors in it
chemical properties of minerals eg are counted on in the
carrying out of metallurgical processes with much the same
certainty and calculable effect as are the motions of those
mechanical appliances by whose use the minerals are handled the
sequence of the process involves both the one and the other both
the apparatus and the materials in such intimate interaction
```

Generating the key:

```
C:\Users\Qi Hao\OneDrive\Desktop\School Assignment\CSCI361\Assignment 1\Task 4>python Kamasutra.py -k keyfile.txt
Key generated and saved to keyfile.txt
```

Keyfile.txt:

laq
bl
cr
ds
ex
fy
gw
ho
ip
jk
kj
lb
mn
nm
oh
pi
qa
rc
sd
tu
ut
vz
wg
xe
yf
zv

Encrypt to create Ctext-3.txt:

```
C:\Users\Qi Hao\OneDrive\Desktop\School Assignment\CSCI361\Assignment 1\Task 4>python Kamasutra.py -e keyfile.txt Ptext-1.txt Ctext-3.txt  
Encryption complete. Ciphertext saved to Ctext-3.txt
```

Ctext-3.txt:

tmfhcutmquxbf rcpuprd daf uox rhnihmxmud qcx hfuxm rbtxs
pmuh uox ichrxdd qms oqzx lxrhn pmuxwcqb fgruhcd pm pu
roxnprqb ichixcupxd hf npmxcqbd xw qcx rhtmuxs hm pm uox
rqccfpmw htu hf nxuqbbtcwprqb ichrxddxd gpuo ntro uox dqnz
rxcuqpmuf qms rqbrtbqlbx xffxru qd qcx uox nhuphmd hf uohdx
nxroqmpqb qiibpamrxd lf gohdx tdx uox npmxcqbd qcx oqmsbxs uox
dxatxmrz hf uox ichrxdd pmzhibzxd lhuo uox hmx qms uox huoxc lhuo
uox qiiqcqutd qms uox nauxcpqbd pm dtro pmupnqux pmuxcgruphm

Decryption of Ctext-3.txt

```
C:\Users\Qi Hao\OneDrive\Desktop\School Assignment\CSCI361\Assignment 1\Task 4>python Kamasutra.py -d keyfile.txt Ctext-3.txt Output.txt
Decryption complete. Plaintext saved to Output.txt
```

```
C:\Users\Qi Hao\OneDrive\Desktop\School Assignment\CSCI361\Assignment 1\Task 4>fc Ptext-1.txt Output.txt
Comparing files PText-1.txt and OUTPUT.TXT
FC: no differences encountered
```

Output.txt:

Unfortunately critics say the components are often clued into the process and have become integral factors in it. Chemical properties of minerals eg are counted on in the carrying out of metallurgical processes with much the same certainty and calculable effect as are the motions of those mechanical appliances by whose use the minerals are handled. The sequence of the process involves both the one and the other both the apparatus and the materials in such intimate interaction

Statistical analysis:

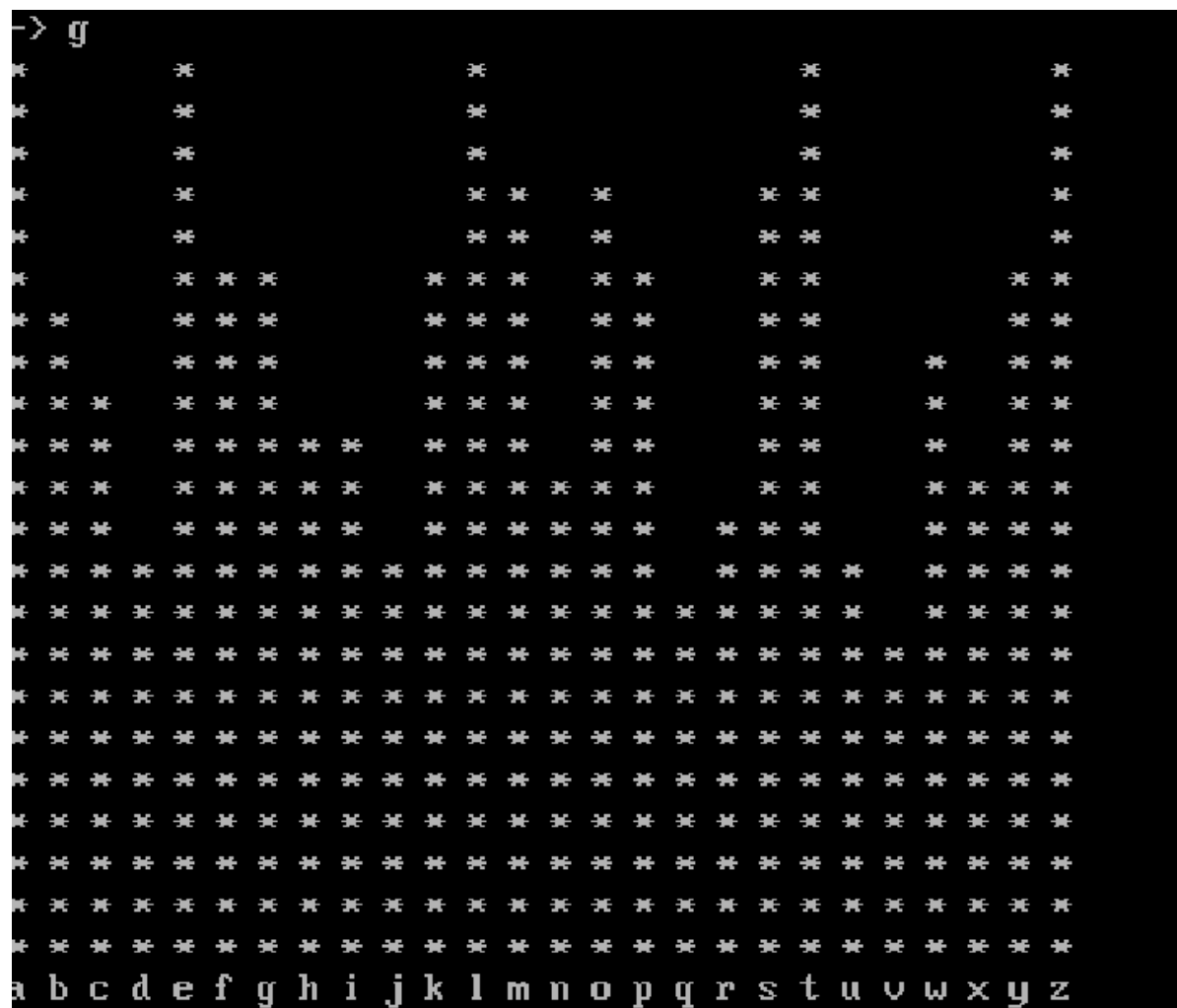
Frequency analysis of Ctext-3.txt:

[illegible]

Frequency analysis of Ctext-1:



Frequency analysis of Ctext-2:



By comparing statistical properties:

For ctext-1:

Frequency distribution: Same shape as plaintext, just shifted

Index of Coincidence (IC): Should be ~ 0.065 because it's a simple substitution

Letter frequencies: Preserved exactly, just rotated

Graph: Will look identical to English frequency distribution

For ctext-2:

Frequency distribution: Flattened/smoothed out

Index of Coincidence: Lower than English (closer to 0.038 for random text)

Letter frequencies: More uniform across letters

Graph: Much flatter than English

For ctext-3:

Frequency distribution: The same as plaintext (Ptext-1)!

Index of Coincidence: Same as plaintext (~ 0.065)

Special feature: Letter 'f' frequency is higher because: Original 'f' stays 'f' and whatever letter is paired with 'f' also becomes 'f' in ciphertext

Graph: Identical to plaintext frequency distribution

Comparison:

Ctext-1 (Caesar) and Ctext-3 (Kamasutra) have identical frequency distributions to the original plaintext.

Ctext-2 (Vigenère) has a flattened distribution (more uniform).

Index of Coincidence:

Ctext-1: ~ 0.065 (English-like)

Ctext-2: ~ 0.040 - 0.045 (lower, polyalphabetic)

Ctext-3: ~ 0.065 (English-like, same as plaintext)

In conclusion: The frequency distribution of Ctext-3 closely matches that of Ctext-1, both showing the characteristic English language pattern with E, T, A being most frequent. In contrast, Ctext-2 shows a nearly uniform distribution across all letters.